

LAB 13 : HASH

Due Saturday at 5:00 p.m. MST

The twelfth and last programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) `std::unordered_set` class.

Unordered Set

Your list class will be in `hash.h` and be named `custom::unordered_set`. It will need to support the following operations:

- **Construct:** default constructor, copy constructor, move constructor, and range constructor.
- **Assign:** copy assign, move assign, initializer list assign, and swap.
- **Iterator:** begin and end.
- **Access:** find and bucket. Note that we did not implement `list::find()` in Lab 05. This gives you two choices: Either perform a linear search in `unordered_set::find()`, or implement `list::find()` and submit your improved `list.h`.
- **Insert:** insert one element and insert from initializer list.
- **Remove:** clear and erase one item.
- **Status:** size, empty, bucket count, and bucket size.

Iterator

Additionally, we will be creating an unordered set iterator. This will be a forward (only increment is needed), read-only iterator. It will need to support the following operations:

- **Construct:** default constructor, copy constructor, non-default construct (so `begin()` and `end()` can properly initialize it).
- **Assign:** copy assign.
- **Compare:** equal-to and not-equal-to.
- **Access:** dereference (read-only).
- **Arithmetic:** prefix increment and postfix increment.

Three Options

This week, there will be three options.

85 Option : Trivial

The 85 option will utilize the **Trivial** implementation as described in the textbook. This will use a fixed array of integers, the array containing exactly ten elements. There are two member variables for this implementation:

```
// exactly 10 buckets of integers.
// note that buckets[iBucket] == HASH_EMPTY_VALUE means it is not filled
int buckets[10];

// number of elements in the hash
int numElements;
```

Notice how we indicate that a bucket is empty with `HASH_EMPTY_VALUE`. For this option, the hash function will be `std::abs()`.

To complete the 85 option, please start with the following files: [232.13.Lab.85.zip](#).

100 Option : Chaining

The 100 option will utilize the **Chaining** implementation utilizing a fixed number of buckets. This will utilize an array of lists as the underlying data structure. The base-type of each element will be specified by a template parameter. There will be two member variables:

```
// exactly 10 buckets
custom::list<T> buckets[10];

// number of elements in the hash
int numElements;
```

Every hash application uses a hash function. The hash function for the 100 option is the default hash function provided by C++

```
size_t hashValue = std::hash<T>{}(t);
```

To complete the 100 option, please start with the following files: [232.13.Lab.100.zip](#).

115 Option : Load Factor

The 115 option will allow for an arbitrary number of buckets. Initially, it will start with 8 buckets but that number will change according to the load factor and to explicit actions by the client. In addition to the interfaces of the 100 option, the following will need to be supported:

- **Construct:** non-default constructor specifying the number of buckets
- **Insert:** rehash and reserve
- **Status:** load factor and max load factor (both the getter and the setter)

The client will also be able to specify the underlying data type, the hash function, the equal-to predicate, and the custom allocator. This means there are four template parameters:

```
template <typename T,
          typename Hash = std::hash<T>,
          typename EqPred = std::equal_to<T>,
          typename A = std::allocator<T> >
```

There will be three member variables:

```
// each bucket in the hash
custom::vector<custom::list<T,A>> buckets;

// number of elements in the hash
int numElements;

// the ratio of elements to buckets signifying a rehash
float maxLoadFactor;
```

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

To complete the 115 option, please start with the following files: [232.13.Lab.115.zip](#).

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `hash.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or was not submitted.
Chaining 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Load Factor 15%	115 solution was attempted				The 85 or the 100 solution was attempted